C++test Parallel Automation

Customer Architecture, Licensing, and Deployment Guide

10 September 2026

1 Purpose of the demonstration

This demonstration evaluates whether analyses distributed across multiple Parasoft C/C++test Standard workers can be consolidated onto one C/C++test Professional Automation host. The customer comparison below addresses three existing Docker workers.

The test runs three different C projects concurrently:

Project	Analysis profile	Deliberately seeded issue types
Flight Sensor Statistics	MISRA C 2023	Missing braces, essential-type conversions, arithmetic issues
Audit Text Processor	Flow Analysis	Uninitialized data, null-pointer path, resource leak
Network Command Queue	SEI CERT C	Unsafe string copy, signed overflow, unchecked result

Each analysis has its own source tree, build directory, C++test workspace, process, configuration and report directory. All three processes share one Automation installation, one container and the same network license service.

2 Current three-container design compared with the demonstration

The customer described the current deployment as three C++test Standard analysis workers running in three Docker containers. The supplied artifacts show the implementation of one worker: its launch script, `cpptestcli.properties` and one console log. The comparison below assumes that this worker pattern is instantiated three times. The supplied artifacts alone do not show all three containers running concurrently or prove how many license tokens License Server charges for that topology.

2.1 What the supplied worker does

One current worker:

- Adds `/opt/parasoft/cpptest` to `PATH` and copies a shared `cpptestcli.properties` into the installation.
- Starts one `cpptestcli` process with a maximum Java heap of 16 GiB.
- Uses GCC 8 build data from `compile_commands_merged.json`.
- Retrieves `Flow Analysis Standard` as a DTP-hosted test configuration.
- Requests a custom network-license edition containing C++test, Automation, Static Analysis, Flow Analysis, DTP Publish and compliance-rule features.
- Generates local XML and HTML reports and publishes results to DTP.
- Maintains a `.cpptest` analysis cache and writes a completion marker for Jenkins.

The supplied console log confirms one Docker machine identity, successful Flow Analysis of a production-sized input and successful DTP publication. It does not identify the number of simultaneous license tokens consumed.

2.2 Architecture comparison

```
Current pattern, repeated three times

Jenkins ──┬─> Docker A -> C++test CLI A ─┐
          ├─> Docker B -> C++test CLI B ─┼─> DTP / License Server
          └─> Docker C -> C++test CLI C ─┘

Demonstrated consolidated pattern

Jenkins or dashboard -> One Docker container ──┬─> C++test CLI A ─┐
                                               ├─> C++test CLI B ─┼─> DTP / License Server
                                               └─> C++test CLI C ─┘
```

The consolidated design still performs three analyses. It does not combine source trees into one scan and it is not one `cpptestcli` process switching between projects. It runs three independent C++test JVMs concurrently inside one container. Each process retains its own input, configuration, workspace, cache, console and report.

Also, "Automation server" does not mean that DTP performs the analysis. C++test Professional Automation is the licensed command-line analysis capability installed on the execution host. DTP and License Server remain external services for licensing, shared configurations and result storage. CPU-intensive code and flow analysis still execute in the Docker container.

2.3 License handling

Both designs can use the same network-license workflow:

1. Each `cpptestcli` process starts and requests the configured edition and features.
2. License Server evaluates the request against the purchased entitlement.
3. The process runs only when the required features are valid.
4. DTP publication is a separate operation that requires an entitled `DTP Publish` or Automation capability.

The current properties explicitly enable network licensing and request a `custom_edition` that includes `Automation`. This means the existing worker is already asking License Server for Automation among its features. The names "Standard server" and "Professional Automation server" should therefore not be used as proof of different license consumption. The installed product version, returned edition, enabled features and actual License Server entitlement are the authoritative evidence.

Container count and license count are not necessarily the same:

- Three containers can present distinct Docker machine identities, but the supplied evidence shows only one of them.
- The demonstrated consolidated container presented one Docker machine identity, even though it ran three scanner processes.
- Every scanner process still performs license validation. In the demonstration, all three concurrent processes independently reported that Automation was valid.
- That result proves concurrent authorization on the tested license. It does not prove that only one token was charged.
- Parasoft network licenses can be floating or machine-limited, and the exact accounting can depend on the purchased license model and feature bundle.

Therefore, consolidation must not be justified as "three licenses become one license" without confirmation. The customer should run three simultaneous production-representative scans while the License Server administrator records active sessions or token usage, then obtain confirmation from Parasoft that this use is contractually permitted. If only fewer concurrent tokens are available, `cpptest.license.wait.for.tokens.time` can wait for a token, but a Jenkins concurrency limit or queue is normally more predictable.

Official Parasoft licensing references:

- https://docs.parasoft.com/display/CPPTEST20252/Setting+the+License
- https://docs.parasoft.com/display/CPPTEST20252/License+Settings

2.4 Technological differences

Area	Three containers	One container with three scanners
C++test processes	One per container	Three independent processes in one container
Installation	Usually repeated or mounted three times	One read-only installation volume shared by all processes
Isolation	Container, filesystem and cgroup boundary per analysis	Process and directory isolation inside one cgroup
Resource control	CPU and memory can be limited per container	Container limit covers the sum of all scanners
Scheduling	Jenkins selects three workers/containers	One worker launches processes concurrently and enforces a concurrency limit
Failure scope	A failed container normally affects one analysis	Container or host failure interrupts all three analyses
Maintenance	Images and installations must remain aligned	One image and installation are upgraded once
Scaling	Add or remove worker containers	Increase process concurrency until the host or license limit is reached
Observability	Logs and metrics are naturally separated by container	Dashboard must label every PID, console, workspace and report
DTP	External license, configuration and reporting service	Same external service; no analysis compute moves to DTP

2.5 Changes required for safe consolidation

The current script copies `cpptestcli.properties` into the shared installation before every run. That is acceptable when each container owns its installation, but concurrent jobs must not overwrite one shared properties file. In the consolidated design:

1. Keep the C++test installation and built-in configurations read-only.
2. Pass a separate `-settings` file to each process instead of copying over the installation-wide file.
3. Give every process unique workspace, `.cpptest` cache, build and report directories.
4. Give DTP publications unambiguous `dtp.project`, `build.id` and `session.tag` values. The supplied properties use one project while build and session identifiers are commented out; parallel publication should not be enabled until result identity and merge behavior are tested.
5. Keep credentials outside the image and repository, using Docker secrets or the customer's approved secret store.
6. Enforce a configured concurrency limit in Jenkins or the launcher.
7. Preserve the existing completion-marker behavior per project rather than sharing one marker.

The public demonstration deliberately sets `report.dtp.publish=false`. It proves concurrent analysis and licensing without writing sample findings into DTP. Concurrent publication of real customer results is a separate acceptance test.

A sanitized implementation example is available in `examples/customer-settings/`. It preserves the active license, DTP, analysis, compiler and reporting settings from the supplied file, retains optional settings as comments and demonstrates three independent `-settings` files without modifying the shared installation.

2.6 Resource impact

Consolidation removes duplicated container operating-system layers and simplifies deployment, but it does not remove scanner resource demand. CPU, memory and disk I/O are aggregated on one host and inside one container.

The supplied production script allows up to 16 GiB Java heap for one scanner. Three such processes may request up to 48 GiB of heap, plus native memory, build tools, filesystem cache and container overhead. The demonstration used only 1 GiB maximum heap per scanner because its sample projects are small. Its resource measurements must not be extrapolated directly to the customer's production analysis.

Before migration, run the actual three workloads concurrently and record peak memory, CPU, disk I/O, analysis duration and license usage. Size the consolidated container for the combined peak with operational headroom, or reduce concurrency and queue excess work.

2.7 Customer conclusion

The demonstration proves that one C++test Professional Automation installation in one Docker container can host three isolated, concurrent C++test analyses. The principal benefit is operational consolidation: one maintained image, one mounted installation, one execution endpoint and one dashboard.

The tradeoff is a larger shared failure and resource domain. It does not inherently reduce the number of simultaneous analysis entitlements, and it requires stronger process-level isolation than the existing container-per-worker pattern. Migration should proceed only after representative load testing, DTP publication validation and written confirmation of the applicable Parasoft license model.

3 What the public demonstration shows

Open:

https://zuwasi.github.io/cpptest-parallel-automation-demo/

GitHub Pages cannot execute commercial C++test software. The public page is therefore clearly marked **Public Replay**. It reconstructs the observed sequence and results so that the interface and parallel workflow can be reviewed without access to the licensed analysis server.

The repository also includes an inspected screenshot from the real Docker run:

https://zuwasi.github.io/cpptest-parallel-automation-demo/docker-live-proof.png

The real live dashboard runs on the analysis host. It reads actual process output and generated XML reports; it does not use replay data.

3.1 How to read the dashboard

- The header identifies replay or live mode, host platform, toolchain and Docker container ID.
- The three project cards show independent status, PID, elapsed time, license state and scan progress.

- Each console window streams the output of its own C++test process rather than a combined log.
- The requirements area explains why a different rule profile is assigned to each project.
- The result area shows files checked, enabled rules and findings parsed from that project's XML report.
- The summary distinguishes successful process completion from the number of static-analysis findings. Findings are expected because each sample intentionally contains defects.

4 Verified result

The reference Docker test used one Ubuntu 24.04 container with C++test Professional 2025.2 and GCC 13.3. The dashboard and all scanner processes ran inside the same container.

The following evidence was independently checked:

- `/.dockerenv` was present inside the dashboard process.
- The dashboard reported the same 12-character container ID as `docker ps`.
- `docker top` showed three C++test Java processes at the same time.
- All three processes independently reported `Automation feature: License is valid`.
- All scanner intervals overlapped for 24 seconds.
- Every process exited with code 0.

Configuration	Files checked	Rules in report	Findings
MISRA C 2023	3/3	385	26
Flow Analysis Standard	3/3	105	2
SEI CERT C Rules	3/3	192	4

During the observed parallel interval, Docker reported approximately 1.67 GiB memory, 467 processes/threads and 849% CPU, equivalent to about 8.5 logical CPU cores. These measurements apply only to the small demonstration projects and must not be used as production sizing values.

5 Repository contents

Repository:

https://github.com/zuwasi/cpptest-parallel-automation-demo

Path	Purpose
`project-alpha/`	MISRA C sample and requirements
`project-beta/`	Flow Analysis sample and requirements
`project-gamma/`	CERT C sample and requirements
`scripts/run-analysis-linux.sh`	Builds and analyzes one project with an isolated workspace
`scripts/run-parallel-test-linux.sh`	Starts three Linux scanner processes and verifies overlap

Path	Purpose
`scripts/run-analysis.ps1`	Equivalent single-project Windows runner
`scripts/run-parallel-test.ps1`	Equivalent Windows concurrency verifier
`server.py`	Live dashboard API, process launcher, console streamer and report parser
`docs/`	Browser dashboard and public replay assets
`Dockerfile`	Ubuntu runtime containing only open-source prerequisites
`compose.yaml`	One-container topology and private volume mounts
`Jenkinsfile`	Windows Jenkins parallel-stage example
`examples/customer-settings/`	Sanitized mapping of the supplied properties and launch process

The repository does not contain C++test binaries, Parasoft configurations, license credentials, DTP credentials or generated customer reports.

6 Reconstructing the Docker demonstration

6.1 Prepare the server

Install the following on a Linux Docker host or a Windows server using Docker Desktop with the Linux container engine:

- Git
- Docker Engine with Docker Compose v2
- A licensed Linux C++test Professional Automation installation
- Network access from containers to the DTP License Server

The tested image uses Ubuntu 24.04, GCC 13 and the C++test `gcc_13-64` compiler profile. Use a compiler/profile pair supported by the customer's installed C++test release.

6.2 Clone the repository

```
git clone https://github.com/zuwasi/cpptest-parallel-automation-demo.git
cd cpptest-parallel-automation-demo
```

6.3 Create the private C++test volume

Create the volume expected by `compose.yaml`:

```
docker volume create cpptest-linux-2025-2
```

Copy the customer's licensed Linux installation into `/opt/parasoft/cpptest` in that volume. For an installation already stored at `/home/customer/parasoft/cpptest`:

```
tar -C /home/customer/parasoft -cf - cpptest \
  | docker run --rm -i \
      -v cpptest-linux-2025-2:/opt/parasoft \
      ubuntu:24.04 tar -C /opt/parasoft -xf -
```

For a Windows server where the source installation is stored in an Ubuntu WSL distribution:

```
docker volume create cpptest-linux-2025-2
cmd /c "wsl.exe tar -C /home/customer/parasoft -cf - cpptest | docker run --rm -i -v cpptest-linux-2025-
        2:/opt/parasoft ubuntu:24.04 tar -C /opt/parasoft -xf -"
```

The volume may contain license configuration and credentials. Restrict Docker access, do not export the volume and never commit its contents.

6.4 Configure licensing

Configure C++test to use the customer's DTP License Server according to the organization's Parasoft deployment policy. Confirm that:

- The container can resolve and reach the license-server hostname and port.
- The selected edition includes Automation, Static Analysis, Flow Analysis, MISRA and CERT features required by the jobs.
- The purchased license terms permit the intended concurrent use.
- `parasoft.eula.accepted=true` is supplied only after the organization has accepted the applicable EULA.

Do not place passwords or license-server credentials in the repository, Dockerfile or Compose file. Keep them in the private installation volume or an approved secret-management mechanism.

6.5 Build and start the container

```
docker compose up -d --build
```

Open the live dashboard on the Docker host:

http://localhost:8876

The header must show `LIVE SERVER`, `C++test Pro 2025.2`, `GCC 13.3 / CMake`, and `Docker` followed by the container ID.

6.6 Run the demonstration

Select **Start parallel scan**. The expected progression is:

1. All three cards enter `running` state.
2. Each card receives a distinct PID.
3. License status changes from `waiting` to `valid`.
4. Three consoles stream different configurations and source files.
5. Every card completes and exposes its generated report.

6.7 Verify Docker independently

Run these commands while the dashboard shows all three analyses as running:

```
docker ps --filter name=cpptest-parallel-demo
docker top cpptest-parallel-demo -eo pid,ppid,comm,args
docker stats cpptest-parallel-demo --no-stream
docker inspect cpptest-parallel-demo
```

Acceptance evidence should include:

- One container named `cpptest-parallel-demo`.
- Three command lines containing `org.eclipse.equinox.launcher.Main`.
- Three different `-data` workspace paths.
- Three different `-config` values.
- Three different `-report` paths.
- Dashboard runtime container ID matching `docker ps`.
- Three valid Automation license messages.
- Three successful exit codes and report files.

6.8 Stop or reset the demonstration

Stop the container without deleting reports:

```
docker compose stop
```

Remove the demonstration container and network while preserving the two named volumes:

```
docker compose down
```

The C++test installation and results volumes are intentionally preserved. Delete them only under the customer's normal data-retention and license-management procedures.

7 Running directly on Linux without Docker

If containerization is not required, clone the repository on the Automation host and run:

```
export CPPTEST_HOME=/opt/parasoft/cpptest
export CPPTEST_COMPILER_FAMILY=gcc_13-64
export OUTPUT_ROOT=/var/tmp/cpptest-parallel-demo
bash scripts/run-parallel-test-linux.sh
```

The script returns failure unless all three intervals overlap, all Automation licenses validate, every report can be parsed and every scanner exits successfully. The machine-readable result is written to `$OUTPUT_ROOT/parallel-test-result.json`.

8 Jenkins integration

The included `Jenkinsfile` demonstrates three parallel Windows branches. For the Docker topology, use a Jenkins agent with Docker access and treat the single container as the Automation execution service:

1. Check out the repository.
2. Run `docker compose up -d --build`.
3. Start analysis with `POST http://localhost:8876/api/run`.
4. Poll `GET http://localhost:8876/api/status` until `running` becomes false.
5. Fail the build unless every project has `status=completed`, `license=valid` and `exitCode=0`.
6. Archive or publish reports from the `cpptest-parallel-results` volume according to the customer's reporting policy.

The reference configuration sets `report.dtp.publish=false` so the proof does not write demonstration results into DTP. Enabling DTP publication should be a separate, controlled integration step using customer project names, build IDs, credentials and retention rules.

9 Production migration considerations

The demonstration proves technical concurrency, not production capacity or contractual entitlement. Before replacing the existing Standard workers:

1. Confirm the Automation license model and concurrency terms with Parasoft or the organization's license administrator.
2. Test representative production codebases, not only the small samples.
3. Measure peak CPU, memory, disk I/O, analysis duration and DTP traffic with two and three concurrent jobs.
4. Establish container CPU and memory limits only after measuring real workloads.
5. Validate compiler profiles, generated-code exclusions, suppressions, baselines and report equivalence.
6. Preserve separate workspaces and report directories for every concurrent job.
7. Define queueing behavior when demand exceeds the safe concurrency level.
8. Add health checks, log retention, backups and monitoring appropriate for the customer environment.
9. Run the old and new systems in parallel during a controlled acceptance period.

The recommended decision criterion is not merely whether three processes can start. Consolidation is ready when representative analyses complete within the required service level, produce equivalent results, remain within licensed concurrency and leave sufficient server capacity for peak demand.